

Algoritmos y Estructuras de Datos II

Trabajo Práctico 1: Complejidad y Método Maestro

Ejercicio 1)

Demuestre que $6n^3 \neq O(n^2)$.

Decimos que una función $f(n)$ pertenece a $O(g(n))$ si existen constantes $c > 0$ y $n_0 > 0$ tales que:

$$f(n) \leq c \cdot g(n) ; \text{ para todo } n \geq n_0$$

Queremos determinar si $6n^3$ pertenece a la clase $O(n^2)$. Por lo tanto, comenzamos suponiendo exactamente eso:

$$6n^3 \in O(n^2)$$

Por lo tanto, deben existir un $c > 0$ y un $n_0 > 0$ tales que

$$6n^3 \leq cn^2 ; \text{ para todo } n \geq n_0$$

Dividimos ambos miembros de la desigualdad por n^2 :

$$6n \leq c$$

pero esto es imposible, dado que, para cualquier n suficientemente grande, $6n$ crece sin límite, superando a c e incumpliendo la desigualdad.

Por lo tanto, se comprueba que $6n^3 \notin O(n^2)$.

Ejercicio 2)

¿Cómo sería un array de números (mínimo 10 elementos) para el mejor caso de la estrategia de ordenación Quicksort(n)?

Dependiendo del pivote elegido, Quick Sort se ejecuta en tiempos distintos. Suponiendo que nuestra implementación de Quick Sort elija como pivote el elemento central del algoritmo, es decir en la posición

$$p = \frac{i+j}{2},$$

donde i es el primer elemento del array y j el último, el mejor array posible sería

$$[0,1,2,3,4,5,6,7,8,9]$$

eligiendo al 4 como nuestro pivote (dada nuestra implementación) y partiendo el arreglo en

$$[0,1,2,3] \mid 4 \mid [5,6,7,8,9]$$

De esta forma, ambos subproblemas ya se encuentran ordenados y al ejecutar recursivamente este orden se mantiene. Para este caso, el algoritmo presenta una complejidad temporal de $\theta(n \cdot \log n)$.

Considerando una lista que no se encuentre ordenada y con otro tipo de mecanismo de elección de pivote, por ejemplo, elegir siempre el primer elemento, otro caso que también tendría la misma complejidad temporal sería un array como el siguiente:

[4,2,1,0,3,7,6,5,8,9]

donde al elegir como pivote al 4, se tiene una distribución similar a la anterior en cantidad de elementos:

[2,1,0,3] | 4 | [7,6,5,8,9]

Ejercicio 3)

¿Cuál es el tiempo de ejecución de la estrategia Quicksort(A), Insertion-Sort(A) y Merge-Sort(A) cuando todos los elementos del array A tienen el mismo valor?

- **Quicksort:** como los elementos son iguales, independientemente del pivote que se tome, todos los elementos quedarán de un solo lado, que es el peor caso para Quicksort. Por lo tanto, la complejidad resulta $\theta(n^2)$. En el caso de que el algoritmo se implemente de forma tal que agrupe elementos iguales, entonces en ese caso puede ejecutarse en $\theta(n)$.
- **Insertion Sort:** el algoritmo debe recorrer el array completo al menos una vez para revisar todos los elementos, pero como los valores son iguales, no debe ordenarlos. Por lo tanto, se ejecuta en $\theta(n)$.
- **Merge Sort:** siempre tiene complejidad $\theta(n \cdot \log n)$, independientemente de la entrada.

Ejercicio 4)

Implementar un algoritmo que ordene una lista de elementos donde siempre el elemento del medio de la lista contiene antes que él en la lista la mitad de los elementos menores que él. Explique la estrategia de ordenación utilizada.

El código correspondiente a este ejercicio se encuentra en el repositorio, en el directorio “code”.

La solución propuesta utiliza un algoritmo de selección conocido como QuickSelect, el cual permite encontrar el k-ésimo menor o mayor elemento de un arreglo o lista. Es decir, cuál es el valor del k-ésimo elemento en una determinada posición objetivo (por ejemplo, de todos los números del arreglo, cuál es el tercero menor).

En este caso, utilizamos QuickSelect para buscar cuál es el elemento central de una lista desordenada y colocarlo en dicha posición. Dado que QuickSelect ordena parcialmente el lado izquierdo o derecho de la lista (dependiendo de cuál sea el pivote elegido), podemos contar cuántos elementos menores que el elemento central se encuentran a la izquierda del mismo.

Esto se realiza en la función *order()*, donde una vez contados estos elementos, intercambiamos la mitad de ellos por elementos mayores al elemento central que se encuentren en la mitad derecha de la lista y así obtenemos que la mitad de los menores se ubiquen a la izquierda y la otra mitad a la derecha.

Este algoritmo se ejecuta en una complejidad temporal promedio de $\theta(n)$. QuickSelect realiza particiones sucesivas, pero solo continúa en uno de los lados. Como el pivote es aleatorio, en promedio, el segmento se reduce a la mitad:

$$T(n) = T(n/2) + n$$

$$T(n) = O(n) + O(n/2) + O(n/4) + \dots$$

Esta suma geométrica converge a

$$T(n) = O(n)$$

Por lo tanto, el caso promedio es $\theta(n)$. Sin embargo, el peor caso teórico es $O(n^2)$, considerando el peor caso de QuickSelect donde se eligen siempre malos pivotes (en este caso, elegir siempre el menor o mayor elemento).

Ejercicio 5)

Implementar un algoritmo *Contiene-Suma(A,n)* que recibe una lista de enteros A y un entero n y devuelve True si existen en A un par de elementos que sumados den n. Analice el costo computacional.

El código correspondiente a este ejercicio se encuentra en el repositorio, en el directorio “code”.

El algoritmo propuesto presenta una complejidad temporal promedio de $\theta(n^2)$. Esto se debe a que, por cada elemento, la lista debe recorrerse nuevamente hasta el final para determinar si ese elemento sumado a algún otro de la lista resulta en el valor n.

Ejercicio 6)

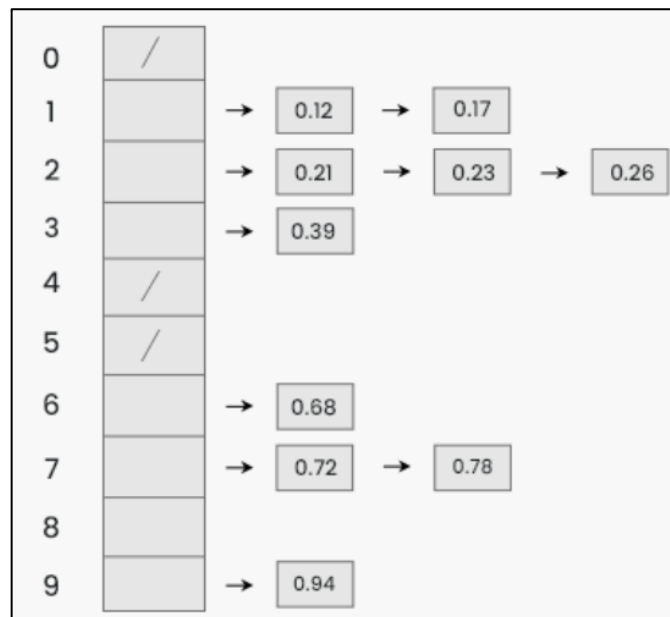
Investigar otro algoritmo de ordenamiento como BucketSort, HeapSort o RadixSort, brindando un ejemplo que explique su funcionamiento en un caso promedio. Mencionar su orden y explicar sus casos promedio, mejor y peor.

Algoritmo elegido: BucketSort

BucketSort es un algoritmo de ordenamiento que consiste en dividir los elementos en distintos grupos o “baldes” (*buckets*). Estos baldes se forman distribuyendo uniformemente los elementos del array. Una vez que todos los elementos están en los baldes, se ordenan utilizando otro algoritmo de ordenamiento (por ejemplo, Insertion Sort). Finalmente, los elementos de cada balde se combinan para obtener el array ordenado.

Ejemplo de funcionamiento en un caso promedio

Supongamos que tenemos el array: [0.78, 0.17, 0.39, 0.26, 0.72, 0.94, 0.21, 0.12, 0.23, 0.68], con un tamaño de 10 elementos. Creamos un nuevo array de baldes, del mismo tamaño, y multiplicamos cada elemento del array original por la longitud del array. Convertimos a entero el resultado obtenido (por ejemplo, $0.23 \cdot 10 = 2.3$, correspondiente a 2) y colocamos los elementos originales en la posición correspondiente del array de baldes. Luego aplicamos un algoritmo de ordenamiento para cada balde individual:



Por último, iteramos sobre cada balde en orden, insertando sobre el array original los elementos ahora ordenados.

Complejidad temporal

En el mejor caso (y también en el caso promedio), donde todos los elementos se distribuyen de manera uniforme en cada balde, el algoritmo presenta una complejidad de $O(n + k)$, que es la suma del tiempo que tarda en ordenarse cada balde sumado al tiempo que tardamos en concatenar los baldes.

En el peor caso, donde todos los elementos quedan en el mismo balde, Bucket Sort se ejecuta en el tiempo correspondiente a la complejidad temporal del algoritmo de ordenamiento utilizado, que en este caso es Insertion Sort, y por tanto resulta en $O(n^2)$.

Ejercicio 7)

a. $T(n) = 2T(n/2) + n^4$

De la ecuación se obtiene que $a = 2$, $b = 2$, $f(n) = n^4$ y por tanto $c = 4$.

$$p = \log_2 2 = 1$$

Por método maestro simplificado, como $p < c$, la complejidad temporal resulta $\theta(n^4)$.

b. $T(n) = 2T(7n/10) + n$

De la ecuación se obtiene que $a = 2$, $b = 10/7$, $f(n) = n$.

$$p = \log_{\frac{10}{7}} 2 \cong 1,94$$

Esta ecuación corresponde al caso 1 del método maestro:

$$f(n) = O(n^{1,94 - \varepsilon}) \Rightarrow \varepsilon = 0,94$$

$$T(n) = \theta(n^{1,94})$$

c. $T(n) = 16T(n/4) + n^2$

De la ecuación se obtiene que $a = 16$, $b = 4$, $f(n) = n^2$ y por tanto $c = 2$.

$$p = \log_4 16 = 2$$

Por método maestro simplificado, como $p = c$, la complejidad temporal resulta $\theta(n^2 \log n)$.

d. $T(n) = 7T(n/3) + n^2$

De la ecuación se obtiene que $a = 7$, $b = 3$, $f(n) = n^2$.

$$p = \log_3 7 \cong 1,77$$

Esta ecuación corresponde al caso 3 del método maestro:

$$f(n) = \Omega(n^{1,77 + \varepsilon}) \Rightarrow \varepsilon = 0,23$$

Verificamos la condición de regularidad:

$$7f\left(\frac{n}{3}\right) \leq cf(n)$$

$$7 \frac{n^2}{9} \leq cn^2$$

$$\frac{7}{9}n^2 \cdot \frac{1}{n^2} \leq cn^2 \cdot \frac{1}{n^2}$$

$$\frac{7}{9} \leq c \Rightarrow c = \frac{7}{9}, c < 1$$

Como la condición de regularidad se cumple, entonces

$$T(n) = \theta(n^2)$$

e. $T(n) = 7T(n/2) + n^2$

De la ecuación se obtiene que $a = 7$, $b = 2$, $f(n) = n^2$ y por tanto $c = 2$.

$$p = \log_2 7 \cong 2,81$$

Por método maestro simplificado, como $p > c$, la complejidad temporal resulta $\theta(n^{2,81})$

f. $T(n) = 2T(n/4) + \sqrt{n}$

De la ecuación se obtiene que $a = 2$, $b = 4$, $f(n) = n^{1/2}$.

$$p = \log_4 2 = \frac{1}{2}$$

Esta ecuación corresponde al caso 2 del método maestro:

$$f(n) = \theta(n^{1/2})$$

$$T(n) = \theta(\sqrt{n} \log n)$$

Si ordenamos las ecuaciones en **orden ascendente** con respecto a su velocidad de crecimiento:

- 1) f. $T(n) = 2T(n/4) + \sqrt{n}$: $\theta(\sqrt{n} \log n)$
- 2) b. $T(n) = 2T(7n/10) + n$: $\theta(n^{1,94})$
- 3) d. $T(n) = 7T(n/3) + n^2$: $\theta(n^2)$
- 4) c. $T(n) = 16T(n/4) + n^2$: $\theta(n^2 \log n)$
- 5) e. $T(n) = 7T(n/2) + n^2$: $\theta(n^{2,81})$
- 6) a. $T(n) = 2T(n/2) + n^4$: $\theta(n^4)$